Language Technology

Chapter 13: Subword Tokenization

https://link.springer.com/chapter/10.1007/978-3-031-57549-5_13

Pierre Nugues

Pierre.Nugues@cs.lth.se

September 14, 2026

Motivation: Language Differences (Source: Xerox)

Breaking a text into morphemes is more economical than breaking it into words.

Language	**# stems**	**# inflected forms**		**Lex. size (kb)**
English	55,000	240,000		200–300
French	50,000	5,700,000		200–300
German	50,000	350,000	or	450
		infinite	(compounding)	
Japanese	130,000	200	suffixes	500
		20,000,000	word forms	500
Spanish	40,000	3,000,000		200–300

Deriving Morphemes

Identifying all the morphemes of every language and writing explicit parsing rules would be extremely difficult.
Instead, morphemes can be derived automatically from a corpus.
Déjean proposed clustering prefixes and suffixes according to the distributional properties of letters.

English	-e -s -ed -ing -al -ation -ly -ic -ent
French	-s -e -es -ent -er -ds -re -ation -ique
German	-en -e -te -ten -er -es -lich -el
Turkish	-m -in -lar -ler -dan -den -inl -ml
Swahili	-wa -ia -u -eni -o -isha -ana -we
	wa- m- ku- ali- ni- aka- ki- vi-

Subword Tokenization Techniques

What if the documents are written in two languages such as Korean and Japanese?
In the context of a multilingual web it is preferable to make as few language-specific assumptions as possible.
We will examine how to

1. create a dictionary of substrings from any raw corpus using statistics only;
2. use these substrings to tokenize a text into subwords.

The main variants of subword tokenization are:

- Byte-Pair Encoding (BPE)
- WordPiece
- Language-model segmentation (unigram)

Overview

These techniques consist of two steps:

1. A **training** step, in which a vocabulary is built (performed once);
2. A **segmentation** step, in which the model (vocabulary) is applied to break a string into tokens.

They are entirely data-driven, although they often rediscover linguistic morphemes.

Benefits

1. The size of the subword vocabulary is a training parameter (e.g. 30 000).
2. Although not compulsory, in machine translation the same vocabulary can be shared between the source and target languages.
3. Sharing the vocabulary can help translate unknown words (for instance proper names) by simply copying the corresponding tokens from the source to the target.

Training

The model and its vocabulary are trained (or extracted) from a corpus. During training:

- We start from the set of characters and successively merge them into longer units until a predefined vocabulary size is reached.
- Merges are decided either by frequency (BPE) or by a language-model criterion (WordPiece).

The resulting vocabulary consists of the most frequent character sequences.

Segmentation

Once the model has been trained, the tokenizer splits a text into subwords by one of the following strategies:

- applying the learned merge sequence (BPE);
- performing a greedy longest match (WordPiece);
- optimising a language-model criterion (unigram model).

SentencePiece implements both BPE and unigram segmentation.

Byte-Pair Encoding (BPE)

Originally BPE was a data-compression algorithm.
Sennrich et al. adapted it to build automatically a lexicon of subwords from a corpus.
The resulting subwords may be

- a single character,
- a sequence of characters, or
- possibly a whole word.

The size of the lexicon is fixed in advance (e.g. 20 000 tokens).

BPE Algorithm

The main steps of the algorithm are:

1. Split the corpus into individual characters. These characters constitute the initial vocabulary.
2. Then repeatedly:
   1. extract the most frequent adjacent pair of symbols in the corpus;
   2. merge the pair and add the new symbol to the vocabulary;
   3. replace every occurrence of the pair in the corpus by the new symbol;
   4. stop when the desired vocabulary size has been reached.

At tokenization time the merge rules are applied in the same order.
BPE can operate on characters or on bytes.
When bytes are used there are, by construction, no unknown tokens: the algorithm always falls back to a single byte.

Demonstration

SentencePiece with BPE:
https://github.com/google/sentencepiece/blob/master/python/sentencepiece_python_module_example.ipynb
Vocabulary file: vocab.m

Pretokenization

BPE normally does not cross whitespace boundaries.
A whitespace pretokenization may or may not be applied:

- The BPE implementation used in GPT-2 performs pretokenization. This speeds up learning because a simple word count can be used. The simplest pretokenization splits on whitespace:

  ```
  pattern = r'\p{L}+|\p{N}+|[^\s\p{L}\p{N}]+'
  ```

  See also: `https://github.com/karpathy/minGPT/blob/master/mingpt/bpe.py`
- SentencePiece works on raw text. It replaces every space by the special character ▁ (U+2581) and never merges across this symbol.

Results

SentencePiece tokenizes the sentence
This is a text

as

```
['▁This', '▁is', '▁a', '▁t', 'est']
```

The ▁ (U+2581) prefix makes the output more readable.
When a pretokenization step is used (as in GPT-2), only the already segmented words are further subdivided.
A cache can be employed to accelerate the successive merges.
Because the resulting tokens can be hard to read, the GPT-2 BPE implementation prefixes non-initial subwords with Ġ.

WordPiece

WordPiece follows principles similar to those of BPE.
The two main differences are:

1. the criterion for merging a pair is the improvement of a language model;
2. tokenization is performed by a greedy longest-match algorithm.

Google has never released the exact algorithm used to construct the WordPiece vocabulary.
If a unigram language model is assumed, the pair that maximises the difference

$$\prod_{i=1}^{N} P(x_i)$$

before and after the merge is selected.
Because no official implementation is publicly available, the vocabulary is often built with BPE instead.

WordPiece Language Model

Before a merge the log-likelihood of the corpus is

$$\sum_{i=1}^{N} \log P(x_i) = \sum_{\substack{i=1 \\ x_i x_{i+1} \neq xy}}^{N} \log P(x_i) + C(xy)\,(\log P(x) + \log P(y)).$$

After the pair xy has been merged it becomes

$$\sum_{\substack{i=1 \\ x_i x_{i+1} \neq xy}}^{N} \log P(x_i) + C(xy) \log P(xy).$$

The difference between the two log-likelihoods is therefore

$$C(xy) \cdot \big(\log P(xy) - \log P(x) - \log P(y)\big).$$

WordPiece Tokenization

Given a vocabulary, WordPiece applies a greedy longest-match strategy.
Example:

- Word: *there*
- Vocabulary: [th, the, he, re, t, h, r, e]

Possible segmentations include [th, e, r, e], [t, he, re], [th, e, re], [the, re], ...
Algorithm:

1. sort the subwords of the vocabulary by decreasing length;
2. build a regular expression that is the disjunction of all subwords;
3. apply `finditer()`;
4. verify that no characters have been lost (see the accompanying demo).

WordPiece marks every non-initial subword with the prefix ##:

```
there  -->  [the, ##re]
```

BERT implementation: `https://github.com/google-research/bert/blob/master/tokenization.py`

Unigram

The unigram tokenizer is concerned only with the segmentation step.
Given a vocabulary, neither longest match nor the original BPE merge order necessarily optimises a language-model objective.
Unigram tokenization therefore selects the segmentation that maximises the product of probabilities (or the sum of log-probabilities):

$$
\begin{aligned}
P(\text{th})\,P(\text{e})\,P(\text{r})\,P(\text{e}) \quad &\text{or} \quad \log P(\text{th}) + \log P(\text{e}) + \log P(\text{r}) + \log P(\text{e}),\\
P(\text{t})\,P(\text{he})\,P(\text{re}) \quad &\text{or} \quad \log P(\text{t}) + \log P(\text{he}) + \log P(\text{re}),\\
P(\text{th})\,P(\text{e})\,P(\text{re}) \quad &\text{or} \quad \log P(\text{th}) + \log P(\text{e}) + \log P(\text{re}),\\
P(\text{the})\,P(\text{re}) \quad &\text{or} \quad \log P(\text{the}) + \log P(\text{re}).
\end{aligned}
$$

A naïve exhaustive search has exponential complexity.
In the laboratory session you will use Norvig's implementation of the Viterbi algorithm:
`https://nbviewer.org/url/norvig.com/ipython/How%20to%20Do%20Things%20with%20Words.ipynb` (Section 5).
Only minor adaptations of his code are required.

Unigram Vocabulary Construction

The vocabulary is usually obtained by a BPE-like procedure.
Token probabilities are then estimated by the expectation-maximisation algorithm:

1. initialise the distribution from a BPE segmentation;
2. resegment the corpus with the current unigram model;
3. re-estimate the token probabilities from the new segmentation;
4. repeat steps 2 and 3 until convergence.

The original unigram paper also includes a pruning step that discards less useful subtokens (see the laboratory session).

SentencePiece

SentencePiece is a subword tokenizer that offers two segmentation algorithms: BPE and a unigram language model.
It treats the space character like any other character, which makes it especially suitable for Chinese, Japanese and Korean.
As a preprocessing step every space is replaced by the special symbol ▁ (U+2581).
SentencePiece is used in many large language models.

Code

- BPE: `https://github.com/rsennrich/subword-nmt`
- SentencePiece: `https://github.com/google/sentencepiece`
- High-performance Rust implementation: `https://github.com/huggingface/tokenizers`